

Activity 10 - Parse CSV records with Result and custom errors

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K4/A4

Specification

`parse_record(&str) -> Result<Record, ParseError>` with `ParseError::`{`FieldCount(n)`, `EmptySku`, `BadNumber{field, value}`} and a `Display` message for each. `parse_all(&str)` skips blank lines and the header, returning valid records and (1-based line number, error) pairs.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain `Result`, the `?` operator, custom error enums, `Display` and `Box<dyn Error>` to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_result_and_question_mark.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on `Result`, the `?` operator, custom error enums, `Display` and `Box<dyn Error>` with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles error handling with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `parse_record(&str) -> Result<Record, ParseError>` with `ParseError::`{`FieldCount(n)`, `EmptySku`, `BadNumber{field, value}`} and a `Display` message for each. `parse_all(&str)` skips blank lines and the header, returning valid records and (1-based line number, error) pairs. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Parse CSV records with Result and custom errors" in `starter/src` so that the supplied tests pass. Return `Result` everywhere; no `unwrap()`, `expect()` or `panic!` in library code. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_result_and_question_mark.rs

Run: rustc --edition 2021 01_result_and_question_mark.rs && ./01_result_and_question_mark

```
// Sample: Result and the ? operator.
use std::num::ParseIntError;

fn parse_qty(text: &str) -> Result<u32, ParseIntError> {
    text.trim().parse::<u32>()
}

fn total_qty(a: &str, b: &str) -> Result<u32, ParseIntError> {
    let first = parse_qty(a)?; // returns early on Err
    let second = parse_qty(b)?;
    Ok(first + second)
}

fn main() {
    println!("{:?}", total_qty("4", " 6 "));
    match total_qty("4", "six") {
        Ok(total) => println!("total {total}"),
        Err(error) => println!("error: {error}"),
    }
}
```

Expected output:

```
Ok(10)
error: invalid digit found in string
```

02_custom_error.rs

Run: rustc --edition 2021 02_custom_error.rs && ./02_custom_error

```
// Sample: a custom error type with Display, used with ? in main.
use std::fmt;

#[derive(Debug)]
enum StockError {
    Unknown(String),
    Insufficient { available: u32, requested: u32 },
}

impl fmt::Display for StockError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            StockError::Unknown(sku) => write!(f, "unknown SKU {sku}"),
            StockError::Insufficient {
                available,
                requested,
            } => {
                write!(
                    f,
                    "requested {requested} but only {available} available"
                )
            }
        }
    }
}
```

```

}

impl std::error::Error for StockError {}

fn ship(sku: &str, available: u32, requested: u32) -> Result<u32, StockError> {
    if sku != "ABC-0001" {
        return Err(StockError::Unknown(sku.to_string()));
    }
    if requested > available {
        return Err(StockError::Insufficient {
            available,
            requested,
        });
    }
    Ok(available - requested)
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    println!("left: {}", ship("ABC-0001", 5, 2)?);
    for (sku, requested) in [("ZZZ-9999", 1), ("ABC-0001", 9)] {
        if let Err(error) = ship(sku, 5, requested) {
            println!("error: {error}");
        }
    }
    Ok(())
}

```

Expected output:

```

left: 3
error: unknown SKU ZZZ-9999
error: requested 9 but only 5 available

```

Learn more

Rust Book: Recoverable Errors with Result:

<https://doc.rust-lang.org/book/ch09-02-recoverable-errors-with-result.html>

Programiz: Rust Error Handling: <https://www.programiz.com/rust/error-handling>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php